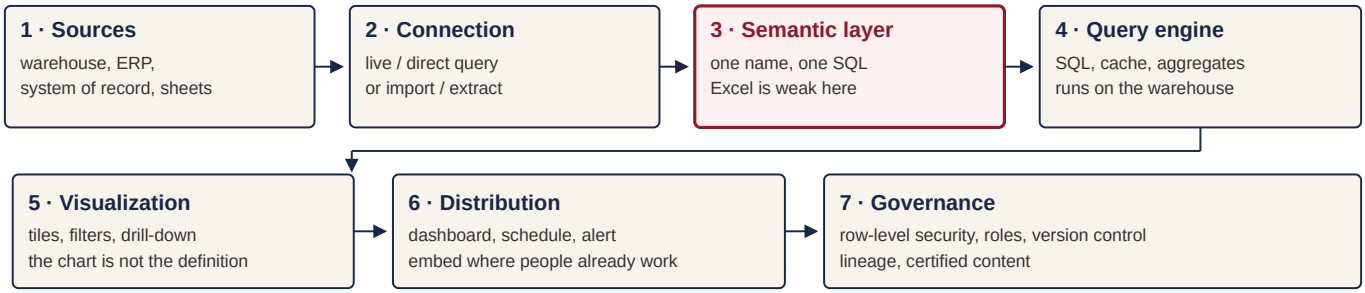
APPLE CAPEX · 30-MINUTE CALL · PART 3 · SAY THIS, DO NOT CLAIM YOU BUILT IT

How a BI tool works, using Looker as the example

> **Call.** CapEx Systems Onboarding Data Integrity Program Manager, with the Capital Expenditure Systems Lead. Thirty minutes. Tableau is a nice-to-have; Power BI or Tableau is preferred. Explain the tool. Do not pretend to be the BI engineer.
> **How to use it.** Section 1 is the model. Section 2 is Looker, only as deep as a follow-up. Section 6 is what you say, including the honest proficiency line.

1. The seven layers of any BI tool

Looker, Tableau, Power BI, and a careful spreadsheet are the same stack. The chart is layer 5. Integrity is decided in the semantic layer and in governance, and it is lost in the connection when nobody knows how fresh the copy is.



LAYER	WHAT IT IS	WHERE INTEGRITY BREAKS
1 · Sources	The warehouse, the ERP, the system of record, and the spreadsheet a team still keeps.	A tile with no source system cannot be reconciled to a PO or an asset.
2 · Connection	**Live / direct query** asks the warehouse each time: fresh, as fast as the source, and the number is the source's number. **Import / extract** copies rows and refreshes on a schedule: fast, and only as fresh as the last refresh.	Trust follows the refresh. A missed refresh is a silent old truth. People believe the tile, not the timestamp.
3 · Semantic layer	One definition per metric. Joins and grain live here. A **dimension** is what you group or filter by (site, program, owner). A **measure** is the aggregation (count, sum, a ratio).	Two formulas with the same name. This is the layer a data-integrity program exists to own.
4 · Query engine	Turns the picked fields into SQL, checks a result cache, and may answer from a pre-aggregate instead of the raw rows.	A cache with no invalidation trigger still serves yesterday after the load has finished.
5 · Visualization	Tiles, filters, drill-down from the number to the rows that made it.	A filter default can hide the exceptions the scorecard is for.
6 · Distribution	The dashboard, a scheduled delivery, an alert when a tile crosses a threshold, an embed in the tool people already open.	Two emailed files diverge the moment someone edits a formula in one of them.
7 · Governance	Row-level security, roles, version control, lineage from the tile back to the column, and a certified mark on the content people should trust.	Without lineage you cannot answer why a field was filled in. That question is the job.

> **Excel is weak at the semantic layer.** A formula in a cell binds nobody. Copy the workbook and the definition forks — five files, five meanings of reconciled — with no row-level security that survives a forward, no lineage back to the source column, and no certified version. Excel can hold rows during a migration. It cannot stop the next copy from changing the rule.

2. Looker specifically

> **What you are allowed to claim.** You can explain this. You have not shipped LookML. Section 6 has the sentence. Looker here means Looker on Google Cloud, the product with LookML. **Looker Studio** (formerly Data Studio) is a different, lighter product: reports on top of connectors, no LookML, no Git-backed semantic model. If they say “Looker,” ask which one in the first sentence, then talk about this one.

Looker does not store the business data. At query time it generates SQL and runs it on the warehouse you point it at — BigQuery, Snowflake, Redshift, and the usual others. What Looker itself keeps is the LookML, the saved content, users and permissions, and a result cache. A persistent derived table is still a table in the warehouse; Looker builds and refreshes it there. The book of record never moves into Looker.

LOOKML OBJECT	WHAT IT IS, IN THE ORDER YOU MEET IT
Project	A Git repository of LookML. This is the unit you branch, review, and merge.
Model	Which warehouse connection, which explores are exposed, and who may open them.
View	One table, or one derived table, plus its dimensions and measures. The definition of a column and of a metric lives here.
Explore	A starting view plus its joins. This is the self-serve surface: the field picker a GSM or a finance partner actually uses. They never see the SQL unless someone opens the SQL tab.

What the model contains

PIECE	THE SENTENCE YOU NEED
Dimension	An attribute. Site, program, supplier, a status. You group by it and filter on it. You do not sum it.
Measure	An aggregate: count, sum, count_distinct, or a ratio written sql: ${a} / NULLIF(${b}, 0). count_distinct on the primary key is the fan-out-safe habit. A raw count counts joined rows.
Dimension group	One timestamp becomes a date, a week, and a month (type: time, with timeframes). Aging buckets are built on that date, not retyped in every tile.
Join	Declared on the explore, with a cardinality: one-to-one, many-to-one, one-to-many, many-to-many. Looker can rewrite a sum so a one-to-many join does not double-count, but only if the cardinality is declared correctly. A ratio you write by hand is not protected for you. Wrong cardinality, wrong total.
Derived table	A view defined by SQL you write, or by a native derived table (Looker builds the SQL from an explore). Use it when the grain you need is not a raw table — one row per line per field, for completeness.
PDT	A persistent derived table: the same idea, materialized in the warehouse and rebuilt on a trigger (a datagroup, often “rebuild when max(loaded_at) changes”). It is a pre-aggregate for speed. It is still warehouse data. A PDT that does not rebuild is a stale extract with a better name.
Aggregate awareness	You declare rollup tables. Looker uses the smallest rollup that can answer the question, and falls back to the base table when you ask for a field the rollup does not have. That is how a weekly total stays fast and a drill to the PO line still works.
Caching	Identical SQL can be served from the result cache. Tie the cache to the same datagroup as the load. No trigger means a Monday tile can contradict a query run a minute later.

What a person touches, and what makes the definition auditable

An **Explore** is the self-serve surface. A **Look** is one saved query. A **dashboard** is a layout of tiles; each tile is a query, not a picture. Filters narrow a tile. drill_fields are the columns you land on when you click a number. A link opens the same record in the system of record — the PO, the asset — so the tile is not a dead end. A scheduled delivery sends the dashboard on a clock (Monday 07:00). An alert fires when a tile crosses a threshold, which a schedule does not do. A Looker Action sends rows onward (a ticket, a webhook) when a person or a schedule asks it to.

Row-level security is a **user attribute** plus an access_filter. The attribute (the program ids this GSM or this CM may see) is set on the user. The filter is injected into the SQL and the user cannot remove it. Content sits in

folders; roles (viewer, explorer, developer, admin, or a custom role) decide who may look, explore, or edit LookML. None of that is a substitute for warehouse permissions. It is the application of them.

LookML in Git is the audit trail. A developer branches, Looker validates the LookML, a pull request is reviewed, and production is the merge. The commit is who changed the definition of reconciled, when, and from what. A Look saved in a personal folder is not that. The metric people steer by has to be the merged one.

```
view: po_lines {
  sql_table_name: capex.po_lines ;;
  dimension: po_line_id {
    primary_key: yes
    type: string
    sql: ${TABLE}.po_line_id ;;
  }
  dimension: site_id {
    type: string
    sql: ${TABLE}.site_id ;;
    description: "Receiving site. Blank means the input owner has not set it."
  }
  dimension: received_date {
    type: date
    sql: ${TABLE}.received_date ;;
  }
  measure: po_line_count {
    type: count_distinct
    sql: ${po_line_id} ;;
    description: "Lines received. Distinct so an asset join cannot double-count."
  }
  measure: null_site_count {
    type: count_distinct
    sql: ${po_line_id} ;;
    filters: [site_id: "NULL"]
    description: "Lines with no site. A completeness exception, not a hidden filter."
  }
  measure: reconciliation_rate {
    type: number
    sql: 1.0 * ${assets.tagged_asset_count} / NULLIF(${po_line_count}, 0) ;;
    value_format_name: percent_1
    description: "Assets tagged / PO lines received. One definition of reconciled."
  }
}

view: assets {
  sql_table_name: capex.assets ;;
  dimension: asset_id { primary_key: yes  type: string  sql: ${TABLE}.asset_id ;; }
  dimension: po_line_id { type: string  sql: ${TABLE}.po_line_id ;; }
  measure: tagged_asset_count {
    type: count_distinct
    sql: ${asset_id} ;;
  }
}

explore: po_lines {
  join: assets {
    type: left_outer
    relationship: one_to_many
    sql_on: ${po_lines.po_line_id} = ${assets.po_line_id} ;;
  }
}
```

Teaching snippet, not a model you have run. Three dimensions on po_lines (po_line_id, site_id, received_date), three measures (po_line_count, null_site_count, reconciliation_rate), one join. The assets view exists only so the ratio has a real numerator. Cardinality is one PO line to many assets; both counts are count_distinct.

3. How one query flows, end to end

1. The user opens the Explore and picks fields — program, site, reconciliation rate — and a filter.
2. Looker reads the LookML and builds the SQL: the columns, the join, the aggregates, and any access filter.
3. It checks the result cache. A hit returns the stored rows and does not touch the warehouse.
4. A miss runs the SQL on the warehouse (or on a PDT, which is still a warehouse table).
5. The warehouse returns the aggregated rows.
6. Looker renders the tile.
7. The user drills. Drill fields return the row-level detail behind the number, still under the same access filter.
8. A link opens that record in the system of record — the PO, the asset — where the input owner can fix the field.

WHERE IT GOES WRONG	WHAT THE INTEGRITY PROGRAM DOES ABOUT IT
Stale extract	A Friday refresh, read on Monday, misses a weekend receipt. The rate looks finished. Require a visible “loaded at,” and treat a cache with no datagroup trigger as the same defect.
Two measures, one name	Both called reconciliation rate. One divides tagged assets by lines received; the other divides by lines ordered. The semantic layer allows one of them in production.
A join that fans out	One line, two assets. A plain count counts the line twice, the denominator inflates, and the rate looks healthier than the book. Declare one-to-many and count distinct on the line id.
A filter default nobody sees	The explore opens on status = Received. Open and in-transit lines never appear, so the scorecard is green because the exceptions were filtered out. Defaults are visible, or they are a bug.
A field with no description	Placed-in-service date with a blank description. Nobody knows whether it means acceptance or the day depreciation starts. The description is the “why” behind the input, written where the field is picked.

4. Power BI, Tableau, and Looker on one table

Same seven layers. The words change. If they say Tableau, stay on this table and do not switch into LookML unless they ask how a definition is audited. If they say Power BI, same table. The integrity idea does not change: one definition, stored where a second copy cannot silently fork it.

	POWER BI	TABLEAU	LOOKER
Where the rows are at query time	Import: copied into VertiPaq, in memory. DirectQuery: still in the source; Power BI sends a query.	Live: still in the database. Extract: copied into a Hyper file and refreshed on a schedule.	Still in the warehouse. A PDT is a table Looker maintains there. The cache holds results, not the book of record.
Modeling language	Power Query (M) shapes the tables. DAX defines the measures.	VizQL draws the sheet. Calculated fields and level-of-detail expressions define the numbers.	LookML. The explore is the only surface a user queries.
Where the metric definition is stored	In the semantic model, the published dataset. A local file can drift from it.	In the workbook, unless someone published a data source and pointed the workbook at it.	In LookML, in Git. A Look stores a query. It does not store the definition.
Row-level security	Roles in the model. A DAX filter decides which rows a role sees.	User filters, or a row-level policy on the published data source.	A user attribute plus access_filter, written into the SQL the user cannot remove.
Versioning	The dataset people query lives in a workspace. Deployment pipelines promote it. Git is optional, not the default audit trail.	Revision history on the server. Certification is a badge a steward applies, not a pull request.	Branch, LookML validation, pull request, merge. The merge is who changed “reconciled,” and when.

	POWER BI	TABLEAU	LOOKER
Best for	A Microsoft shop, a fast in-memory model, apps handed to a large set of viewers.	Visual analysis on a published data source someone has certified, so ten workbooks do not each invent the ratio.	One audited definition, queried live, when the failure mode is five spreadsheets and five formulas.

A calculated field pasted into ten Tableau workbooks is the same fork as five spreadsheets. Publishing the data source and certifying it is how Tableau tries to stop that. LookML makes the stop structural: production is the merge.

5. Worked example — the CapEx data-integrity scorecard in Looker

> **Illustrative, end to end.** This is the scorecard from the product idea, drawn as a Looker explore so you can walk it. It is not Apple's tool, and none of the counts below are a program you ran. Every figure on this page is **ILLUSTRATIVE**.

Tables behind the explore: **programs**, **purchase_orders**, **po_lines**, **assets**, **suppliers**, **sites**, **input_owners**. The explore starts at po_lines, one row per received line. It joins **assets** one-to-many (a line can tag more than one asset — the fan-out), **programs** and **sites** many-to-one, and **input_owners** many-to-one on the line's owner. **purchase_orders** and **suppliers** are many-to-one parents, joined only when a tile needs the PO number or the supplier name. They are not on the fan-out path. Reconciliation measures are count_distinct on the line id and on the asset id. A plain count would count a line once per asset and make the rate look healthier than the book.

Field completeness is a different grain: one row per line per field (placed-in-service date, site, cost center, program allocation, reuse source asset), because each field has its own owner. That derived table is one-to-many off the line. Its measures count distinct lines *inside one field*. Joining it into the reconciliation tile would multiply the denominator. Two tiles, two grains, one explore only if the joins are the safe cardinality.

CapEx data-integrity scorecard · scheduled Monday 07:00 · row-level security on program

EVERY NUMBER IN THIS WIREFRAME IS ILLUSTRATIVE

94.2%
PO ↔ asset reconciliation
113 assets tagged / 120 lines received. Assumes one asset on each tagged line.

5 · 3 · 1
Exception aging, open
0–7 days · 8–30 · 31+. Nine open. The 31+ row is a close-out, not a color.

81%
First-time-right
17 of 21 required fields passed on first submit. Two newly onboarded programs.

3
Person-dependent fields
Edited by one user only in the window. The knowledge leaves with that person.

FIELD	INPUT OWNER	FILLED	NULL	COMPLETE	FLAG
Placed-in-service date	Finance	108	12	90%	One editor — person-dependent
Site	Contract manufacturer	116	4	97%	
Cost center	GSM	111	9	93%	
Program allocation	TPM	102	18	85%	One editor — person-dependent
Reuse source asset	AME	40 of 48 reuse lines	8	83%	One editor — person-dependent

ILLUSTRATIVE throughout, including the 98% alert line below. The picture assumes one tagged asset per tagged line, so "assets tagged / lines received" equals "lines with an asset / lines received." If a line can carry two assets, certify the line version. The asset version then reads high and hides untagged lines. Say that before anyone builds the tile. 113 / 120 = 94.2%; 17 / 21 = 81%; 40 / 48 = 83%.

```
explore: po_lines {
  access_filter: {
    field: po_lines.program_id
    user_attribute: allowed_program_ids
  }
  join: assets {
    type: left_outer
    relationship: one_to_many
    sql_on: ${po_lines.po_line_id} = ${assets.po_line_id} ;;
  }
  join: programs {
    relationship: many_to_one
    sql_on: ${po_lines.program_id} = ${programs.program_id} ;;
  }
  join: sites {
    relationship: many_to_one
    sql_on: ${po_lines.site_id} = ${sites.site_id} ;;
  }
  join: input_owners {
    relationship: many_to_one
    sql_on: ${po_lines.field_owner_id} = ${input_owners.owner_id} ;;
  }
}
```

Each GSM and each CM gets a user attribute, allowed_program_ids. The access filter is in the SQL, and they cannot clear it. A scheduled delivery goes out Monday at 07:00 to the input owners on the completeness tile — the people who can fill the null, not a general list. An alert, separate from the schedule, fires when reconciliation falls below the team's threshold. Ninety-eight percent in the sketch is illustrative; you do not invent the real line. The description on placed-in-service date says whether the field means acceptance or the day depreciation starts. That is the why behind the input, sitting where the field is picked.

What changes in behavior. A null site is the contract manufacturer's row, not a shared inbox. An exception has an age, and the one past thirty-one days is a close-out conversation with an owner. "Reconciled" is the merged measure. A side spreadsheet that divides tagged assets by lines ordered is a different metric, and it cannot replace the tile without a pull request.

---

6. How to say it in the call

SIXTY SECONDS — HOW A BI TOOL WORKS

A BI tool is seven layers, and the chart is only the fifth. The rows stay in the warehouse, the ERP, or the system of record. The connection is either live, meaning the tool asks the source every time, or an extract, a copy that is only as fresh as its last refresh. The layer that protects data integrity is the semantic layer: one name, one formula, the join and the grain written down, so reconciled means one thing on every tile. The query engine turns that into SQL, checks a cache, and sometimes uses a pre-built aggregate. People see a tile, filter it, and drill to the rows. Distribution is the dashboard, the Monday email, and an alert. Governance decides who sees which programs, and whether that definition is versioned. Excel can hold the rows. It does not hold a definition. The next copy of the file is a second meaning of reconciled.

WHAT THE SOURCES ACTUALLY SUPPORT — READ THIS BEFORE YOU IMPROVISE

/tmp/sources was not on disk when this primer was written. There was no LSA workbook, no BRD text, and no WBR deep-dive text to quote. What follows is what the numbers canon records about those files (/workspace/amazon/numbers-reconciliation.md, rows S11.24, S16, S20), not a fresh read of the files. Do not upgrade it.

LSA_WBR_2022, Instructions rows 6–9. A weekly-review format: input to output, a five-to-six-week window, red or green when the week-over-week move is more than 5% relative, week running Monday to Sunday. The canon calls it

a format copied from the LSA review. It is an operating-review convention in a spreadsheet. It is not a Looker model and not a Tableau workbook.

BRD for Zendesk data in Snowflake, pages 1–5. You wrote the requirements. The as-is was manual pulls into spreadsheets, an hour and a half to two hours for one week-over-week snapshot of seven metrics. The document names Looker SQL Runner as the reporting surface and Snowflake as the warehouse, and it prices the manual work. The infrastructure team built the pipeline. You froze the seven definitions and the acceptance gate — a parallel run within 1% for consecutive weeks — and you called the cutover. You did not write LookML. The BRD names no table or column.

WBR deep dive (the Tech Ops prep book the canon tags S20). Your own interview-prep notes. The sample weeks are marked illustrative. Not a dashboard you shipped. Not evidence of Tableau, Power BI, or Looker.

SQL, narrower than "I built the warehouse." The interview workbook's Austria routing-guide story says you pulled three years of shipment history in SQL yourself, to show true data quality near 55% rather than the 40% that had been reported. That is a pull against a source. It is not a semantic layer. The companion packet's reading of the resume is the same shape: Tableau and Power BI do not appear in the mapped experience bullets.

THE PROFICIENCY SENTENCE — SAY THIS, THEN STOP

I would pick up Looker or Tableau quickly; I have learned three enterprise systems to go-to level.

If they ask what you have actually done. "I built the metrics behind the weekly review in spreadsheets, and I wrote the definitions and the parity gate for the Snowflake pipeline. Looker SQL Runner was the surface the requirements named. I did not build the Looker layer." If they ask which three systems, do not count Looker, Tableau, or Power BI. Name systems you can walk: the warehouse system you took through cutover, and SAP. Offer a third only if you can open it from memory. This primer does not freeze that third name.

THE LINE THAT LANDS WITH A DATA-INTEGRITY TEAM

The semantic layer is where you stop five spreadsheets from each having their own definition of reconciled.

Ten terms, if they use one

TERM	ONE SENTENCE
Semantic layer	The place a metric is defined once — name, SQL, grain, join — so every tile uses that definition.
Dimension	Something you group or filter by: site, program, owner, a date. You do not sum it.
Measure	An aggregate: a count, a sum, a count distinct, or a ratio of two of those.
Explore	The self-serve surface in Looker: one starting view, its joins, and the field picker.
LookML	Looker's modeling language. It lives in Git. Production is a merge, which is what makes a definition auditable.
PDT	A persistent derived table: a pre-aggregate Looker builds and refreshes in the warehouse, not inside Looker.
Fan-out	A one-to-many join repeats the left-hand row. A plain count then double-counts. Count distinct on the key, or declare the cardinality and mean it.
Row-level security	A filter the tool applies for you, from who you are. In Looker that is a user attribute plus an access filter. The user cannot take it off.
Extract vs live	An extract is a scheduled copy, fast and possibly stale. Live, or direct query, asks the source at query time and is only as fast as that source.
Certified content	The dataset or dashboard a steward has marked as the one to trust. In Tableau that is a badge on a published source; in Power BI, an endorsement. In Looker the equivalent is LookML merged to production, not a Look in a personal folder. Looker Studio is a different product — do not invent a Looker badge.